

Neural Network Attacks

Adversarial Examples Lab

Machine Learning and Security

Fabio De Gaspari
Dorjan Hitaj

Adversarial Examples - Lab

Adversarial Examples Lab

What to implement:

- FGSM
- IFGMS
- PGD
- Adversarial Training

Model: pre-trained cat vs non-cat binary classifier

Adversarial Examples Lab

FGSM: $\tilde{x} = x + \epsilon \; sign(\nabla_x J(x, \theta))$

Take an ϵ-step in the direction of the gradient of J wrt. x

In pytorch, you can get the gradient in two ways:

- loss.backward(): computes gradients for all tensors with requires_grad=True
  - you can access the gradient using Tensor.grad after calling backward()
- torch.autograd.grad(loss, inputs): compute gradient of loss wrt inputs
  - this is actually more complex than what written here, but you can use it like this for our purposes. For more info see pytorch docs

The code is setup to implement this using loss.backward()

Adversarial Examples Lab

FGSM: $\tilde{x} = x + \epsilon \; sign(\nabla_x J(x, \theta))$

Take an ϵ-step in the direction of the gradient of J wrt. x

The code is setup to implement this using loss.backward()

1. create a Tensor with the perturbations δ initialized as 0
2. predict on the samples x + perturbation δ
3. compute gradient for δ
   - since we sum δ to x in the prediction, the gradient has the same direction for x and δ, so we can just take δ.grad

! Careful: remember that you might have to .detach() tensors from the computation graph in some cases !

Adversarial Examples Lab

IFGSM: $x^{(k+1)} = \mathcal{P}_{[-\epsilon,\epsilon]} \left\{ x^{(k)} + \alpha \, sign(\nabla_{x^k} J(x^{(k)}, \theta)) \right\}$

Iteratively take an α-step in the direction of the gradient of J wrt. x

The code is setup to implement this using torch.autograd.grad()

1. initialize adversarial data;
2. iterate:
    a. predict on adversarial data
    b. compute loss and gradient
    c. step (add perturbation) *this operation should not be in the computational graph*
    d. project perturbation

Adversarial Examples Lab

PGD: $x^{(k+1)} = \mathcal{P}_{[-\epsilon,\epsilon]}\left\{x^{(k)} + \alpha\, sign(\nabla_{x^k} J(x^{(k)}, \theta))\right\}$

Like IFGSM: iteratively take an *α*-step in the direction of the gradient of J wrt. x

- but initialize perturbation randomly within allowed perturbation space
- adds restarts: perform IFGSM from different random starting perturbations multiple times
  - for each sample, keep track of perturbation that maximizes the loss across restarts

Adversarial Examples Lab

Things to keep in mind:

- gradient computation: loss.backward() and torch.autograd.grad()
- projection: torch.clamp(tensor, min, max) or Tensor.clamp(min, max)
- you can take the sign of a Tensor with .sign()
- you can use Tensor.uniform_(min, max) to randomly fill tensor from uniform distribution in [min, max]
- you can create a tensor of the same shape of another using torch.zeroes_like() or torch.empty_like()
- the dataset is normalized: images are /255 -> values are in [0,1]
  - the perturbed images cannot have values < 0 or > 1

Adversarial Examples Lab

We'll use the catvnoncat dataset: binary classification task. The zips include a model pre-trained on this dataset

- You can train your own if you want, see train_model.py

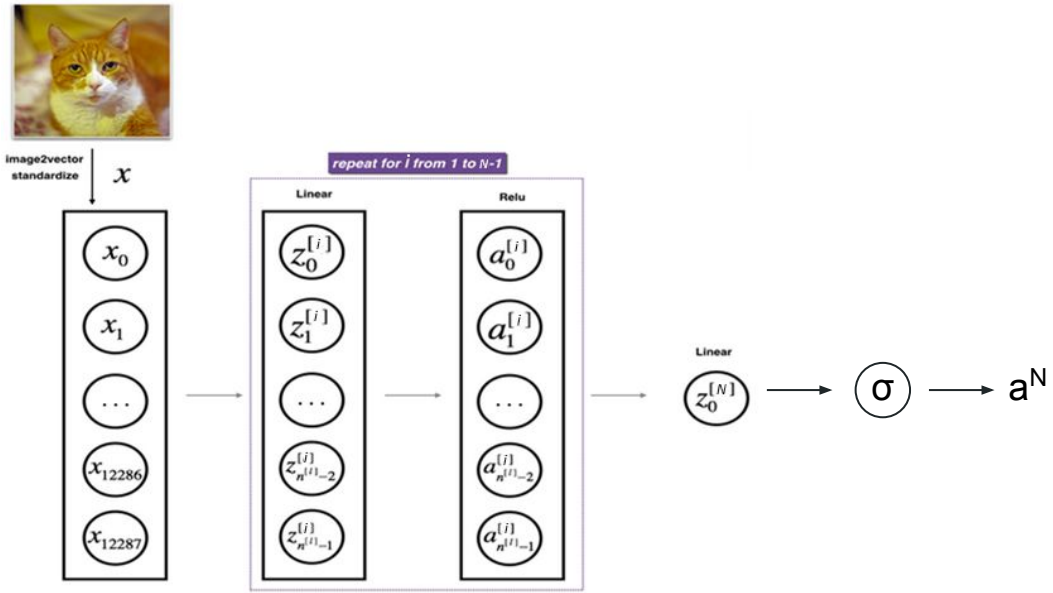


image from deep learning specialization, Andrew Ng

Adversarial Examples Lab

Bonus: the zips also include the MNIST digit recognition dataset and a pre-trained model you can use to generate adv. examples on MNIST

- 10-class classification problem (digits 0 - 9)
- 60k training and 10k testing images
- 28x28 images, single channel. Must be flattened to 1D -> 784 features (pixels)

